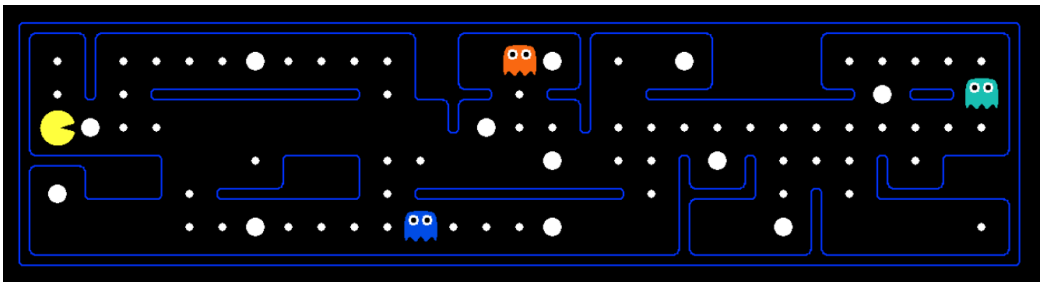
Défis en IA - Apprentissage par renforcement

Xavier Siebert

2023-2024



L'objectif de ces "défis" est de se familiariser avec l'apprentissage par renforcement (*Reinforcement Learning*), concept et outil incontournable dans le domaine de l'intelligence artificielle de nos jours. Ces défis, inspirés de travaux pratiques à l'Université de Berkeley (cours `cs188`), sont divisés en 3 parties:

1. Mise en pratique des concepts de base sur des grilles représentant des labyrinthes simples,
2. Application de ces concepts pour la programmation d'un PacMan,
3. Introduction à l'apprentissage par renforcement profond (*Deep Reinforcement Learning*).

Le code des deux premières parties consiste en plusieurs fichiers en `Python`, qui doit donc être préalablement installé, par exemple en utilisant la distribution `Anaconda` (`www.anaconda.com`).

Pour une application directe de ce défi, seuls les fichiers suivants devront être modifiés:

- `valueIterationAgents.py`

- `qlearningAgents.py`.

Les autres fichiers peuvent être consultés au besoin, mais ne doivent pas forcément être modifiés. De plus, seules certaines sections indiquées dans ces fichiers doivent être modifiées, selon les instructions données.

1 Défi RL1

1.1 Premiers pas dans un labyrinthe

Pour commencer, vous pouvez contrôler "manuellement" (`-m`) à l'aide des flèches sur le clavier un agent (•) dans une grille représentant un labyrinthe simple (`-g BookGrid`), en utilisant la commande

```
python gridworld.py -m -g BookGrid
```

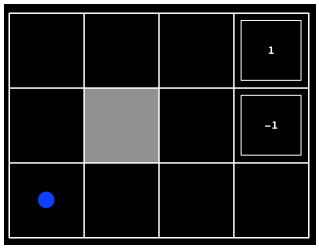


Figure 1: `BookGrid` est la grille par défaut

L'objectif de l'agent est d'atteindre la case 1, sans tomber dans la (-1). Remarquez que l'agent est *stochastique* : quand on tente de le déplacer dans une direction, il ne le fait qu'une fraction du temps (80%, ce qui est contrôlé par la paramètre `--noise`, valant 0.2 par défaut). Ceci est à mettre en relation avec la définition de la probabilité de transition vue dans l'introduction théorique :

$$T : \mathcal{T} \to [0,1] : \{s, a, s'\} \to T(s,a,s') = \mathbb{P}(S_{t+1} = s' | A_t = a, S_t = s).$$

Essayez par exemple d'autres valeurs du paramètre `--noise` pour vous rendre compte de ce caractère stochastique (attention, les valeurs trop élevées deviennent vite agaçantes !).

```
python gridworld.py -m -g BookGrid --noise=0.7
```

D'autres aspects et paramètres de la simulation peuvent être contrôlés, on peut en voir la liste complète :

```
python gridworld.py -h
```

L'agent automatique par défaut exécute aléatoirement les actions:

```
python gridworld.py -g BookGrid
```

Pour essayer une autre configuration de grille :

```
python gridworld.py -g MazeGrid
```

Le processus de décision de Markov (MDP) dans la grille est conçu de façon à ce que l'agent doive entrer dans un état pré-terminal (cases 1 et -1), dans lesquels une seule action est possible pour se rendre dans l'état terminal (non visible dans l'interface graphique). Vous remarquerez donc que la partie se clôture *après avoir quitté* la case 1 ou -1.

1.2 Itération par valeurs

L'objectif est à présent d'implémenter un agent utilisant l'algorithme d'itération par valeurs, comme dans l'équation (1). Cet algorithme calcule v_k, après k estimations de la valeur optimale.

Il s'agit ici d'un MDP où les fonctions $T(s, a, s')$ et $R(s, a, s')$ de l'équation (1) sont données.

$$v_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma v_k(s')] \tag{1}$$

Dans une implémentation de base, il faudra donc prévoir deux tableaux, pour stocker les valeurs de v_{k+1} et v_k, respectivement. Une version "in-place" avec un seul tableau est également possible.

1. Dans le fichier `valueIterationAgents.py`, modifiez et complétez le constructeur (`__init__`) de la classe `ValueIterationAgent`. Ce constructeur reçoit donc en entrée un MDP, le facteur de déprécation γ et le nombre d'itérations de l'équation (1) qui devront être effectuées au sein de ce constructeur.

   Pour vous aider, lisez attentivement les instructions dans chaque fichier; les sections à compléter sont indiquées par

```
"*** YOUR CODE HERE ***"
```

N'oubliez pas de considérer le cas où un état n'a pas d'actions disponibles dans le MDP.

2. Complétez également les 3 méthodes suivantes en utilisant v_k,

   - `getValue(state)` : renvoie la valeur d'un état
   - `getPolicy(state)` : renvoie la meilleure action, étant donné les valeurs calculées
   - `getQValue(state, action)` : renvoie la q-value de la paire (state, action)

   Pour tester votre code avec un `valueIterationAgent`, utilisez l'option "`-a value`", par exemple:

   ```
   python gridworld.py -g BookGrid -a value -i 100 -s 0.5
   ```

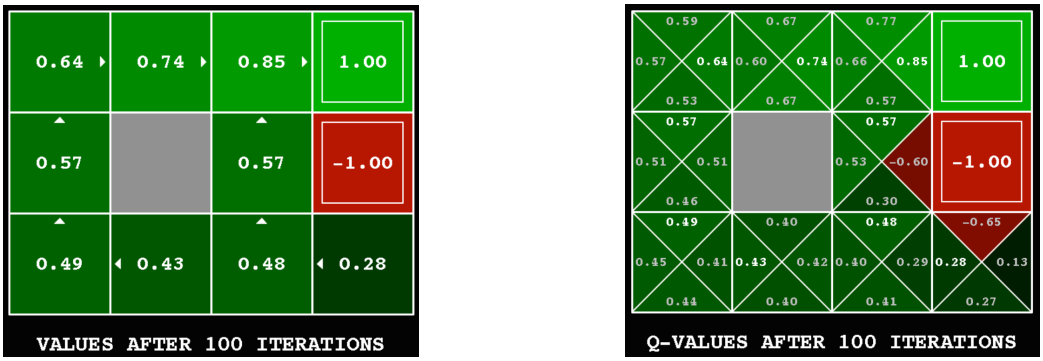


Figure 2: Fonctions de valeurs (gauche) et q-values (droite) pour la grille `GridWorld` après 100 itérations.

Un des paramètres importants est le nombre d'itérations (option `-i`) dans la phase initiale. Le paramètre `-s 0.5` contrôle la vitesse d'affichage, et vaut 1 par défaut.

Les quantités `Value`, `QValue`, `Policy` sont toutes visibles dans l'interface graphique (Fig. 2) : les valeurs sont les nombres dans les carrés à

gauche, les q-values sont les nombres dans les subdivisions triangulaires à droite, et les politiques sont indiquées par des flèches blanches à gauche.

3. Vérifier votre implémentation en effectuant le calcul de la politique après 100 itérations, en le répétant 10 fois (10 "épisodes"):

   ```
   python gridworld.py -a value -i 100 -k 10
   ```

   Commenter les résultats obtenus: les valeurs et les politiques semblent-elles raisonnables ?

1.3 Le pont de la rivière Kwaï

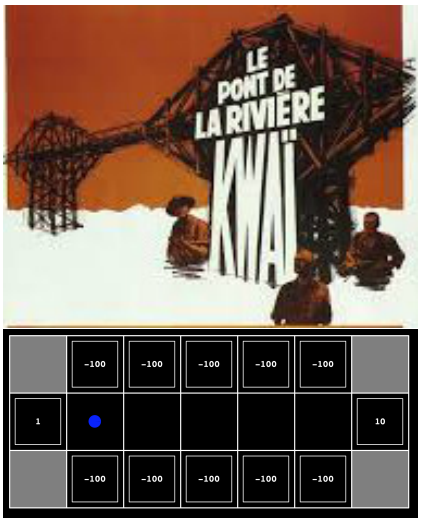


Figure 3: Le pont de la rivière Kwaï et son modèle `BridgeGrid`

La grille `BridgeGrid` est une grille ayant un état terminal avec une faible valeur (1) ainsi qu'un état terminal ayant une valeur élevée (10) séparés par un pont. De part et d'autre du pont se trouve la rivière Kwaï, composée d'états ayant des récompenses négatives. L'agent commence près de l'état terminal à gauche, ayant une faible valeur. Pour rappel, l'agent est stochastique, et le bruit (`--noise`) indique la probabilité que l'agent se déplace dans

une direction différente de la direction voulue en exécutant une action. Il se pourrait donc que l'agent tombe à l'eau alors que l'action consistait à se déplacer sur le pont.

1. Lancez l'algorithme l'itérations par valeurs, avec le facteur de dépréciation (*discount rate*) $\gamma = 0.9$ et le bruit de 0.2.

   ```
   python gridworld.py -a value -i 100 -g BridgeGrid
          --discount 0.9 --noise 0.2
   ```

   Vous devriez obtenir les valeurs de la Fig. 4.



Figure 4: Grille `BridgeGrid` après 100 itérations par valeurs avec $\gamma = 0.9$ et un bruit de 0.2.

   Remarquez que la politique optimale ne parvient pas à traverser le pont, l'agent retourne vers la gauche.

2. Changez un seul de ces paramètres de façon à obtenir une politique capable de traverser le pont et expliquez votre raisonnement.

1.4 Un peu de politique...

Observez la grille DiscountGrid de la Fig. 5. Cette grille possède deux états terminaux avec des récompenses positives (dans la rangée du milieu):

- un situé près de l'état initial (en jaune) avec une récompense de +1
- un autre plus loin avec une récompense de +10.

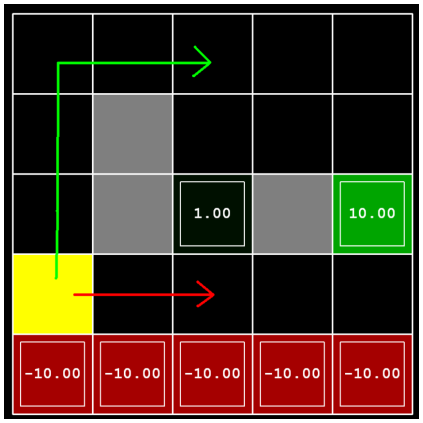


Figure 5: Grille `GridWorld`

Les états dans la rangée du bas de cette grille ont tous une récompense négative de -10.

Il existe deux types de chemins dans cette grille:

- les chemins qui prennent le risque de passer près des états négatifs; ces chemins (flèche rouge sur la Fig. 2) sont plus courts, mais ils risquent d'obtenir une récompense fortement négative.
- les chemins plus sûrs qui font le détour par le haut de la grille; ces chemins (flèche verte sur la Fig. 2) sont plus longs, mais ont moins de chances d'obtenir une récompense fortement négative.

1. Lancez le programme avec les instructions suivantes

   ```
   python gridworld.py -a value -i 100 -g DiscountGrid
            --discount 0.9 --noise 0.2 --livingReward 0.0
   ```

   et remarquez que l'agent va préférer la sortie lointaine (+10) en évitant les états négatifs (-10), comme sur la Fig. 6.

2. Rappelez le rôle de chacun des paramètres : facteur de dépréciation (`discount`), bruit (`noise`) et récompense (`livingReward`).

3. Trouvez les valeurs de ces paramètres afin que le processus de décision de Markov produise des politiques optimales avec les effets suivants, si c'est possible:



Figure 6: Grille `GridWorld`, après 100 itérations de valeurs, avec comme paramètres `--discount 0.9 --noise 0.2 --livingReward 0.0`.

(a) Préférer la sortie proche (+1) en risquant les états négatifs (-10),

(b) Préférer la sortie proche (+1) en évitant les états négatifs (-10),

(c) Préférer la sortie lointaine (+10) en risquant les états négatifs (-10),

(d) Éviter les deux sorties et les états négatifs (de façon à ce qu'un épisode ne se termine jamais).

Vérifiez chaque réponse en adaptant les instructions aux valeurs de paramètres choisis.

1.5 q-learning

L'agent effectuant des itérations par valeurs (`-a value`) n'apprend en réalité pas à partir de son expérience. Il calcule à partir de son modèle MDP une politique complète, avant même d'interagir avec l'environnement. Cette interaction se fait en suivant la politique précalculée (par "réflexe").

Pour des problèmes réels, le MDP complet n'est cependant pas disponible.

1. Dans la classe `QLearningAgent` du fichier `qlearningAgents.py` se trouve une esquisse d'un agent q-learning, qui apprend par "essais et erreurs" lors de ses interactions avec l'environnement.

Complétez l'implémentation de cet agent en modifiant les méthodes suivantes[1]:

- `update`
- `getValue` (`computeValueFromQValues`)
- `getQValue`
- `getPolicy` (`computeActionFromQValues`)

Vérifiez votre implémentation sur `BookGrid`, en utilisant l'option `-a q` au lieu de `-a value` qui était utilisé pour l'itération de valeurs, les autres paramètres `[...]` étant ceux utilisés précédemment :

```
python gridworld.py -a q [...]
```

Quelques remarques :

- Lorsque plusieurs actions ont des valeurs maximales égales dans la méthode `getPolicy`, choisissez le gagnant de manière aléatoire, par exemple à l'aide de la fonction `random.choice()`.
- Dans un état donné, les actions que l'agent n'a pas encore vues ont tout de même une valeur $q = 0$; si toutes les actions que l'agent a vues jusqu'à présent ont une valeur $q < 0$, une action qu'il n'a pas encore vue pourrait être optimale.
- Assurez-vous de toujours accéder aux valeurs q dans les méthodes `getValue` et `getPolicy` en appelant la méthode `getQValue`. Cela sera important pour la suite...

2. Visualiser votre agent q-learning avec un contrôle manuel des étapes, sur 5 épisodes:

```
python gridworld.py -a q -k 5 -m
```

[1]Les noms entre parenthèses correspondent aux versions plus récentes des projets de l'université de Berkeley dont s'inspire ce défi, `https://inst.eecs.berkeley.edu/~cs188/fa18/project3.html`.

Observez que l'agent renforce ses connaissances sur l'état qu'il quitte (s), non pas sur celui vers lequel il se déplace (s').

Conseil: pour déboguer votre code, vous pouvez désactiver le bruit avec le paramètre `--noise 0.0`. Vous devriez alors obtenir, en suivant en mode manuel le chemin {N,N,N,E,E,E,E} à 4 reprises, les valeurs de la Fig. 7.

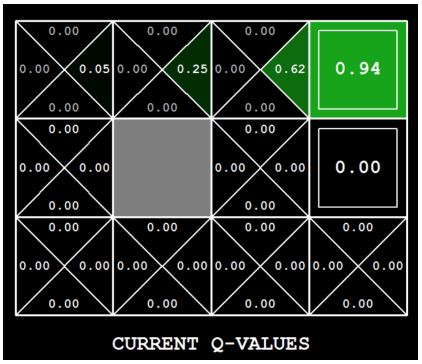


Figure 7: Grille `GridWorld`, après 4 épisodes de q-Learning en mode manuel {N,N,N,E,E,E,E}, avec comme paramètres `--noise 0 -a q -k 5 -m`.

1.6 Compromis exploration/exploitation

1. Complétez votre agent q-learning en implémentant une sélection ϵ-Greedy des actions dans la méthode `getAction`. Autrement dit, votre agent doit choisir une action aléatoire (exploration) avec une probabilité ϵ et utiliser les valeurs q sinon (exploitation). Pour simuler une variable binaire avec une probabilité de succès p, vous pouvez par exemple utiliser la fonction `util.flipCoin(p)`, qui retourne `True` avec une probabilité p et `False` avec une probabilité $(1-p)$. Notez qu'une action aléatoire pourrait être une action optimale; vous devez choisir parmi toutes les actions valides, pas uniquement les actions sous-optimales.

2. Testez votre implémentation

```
python gridworld.py -a q -k 100
```

Vos q-values finales devraient ressembler à celles de l'itération par valeurs, en particulier le long de chemins parcourus souvent. Cependant, les récompenses moyennes seront plus basses que les q-values prédites, à cause des actions aléatoires et de la phase d'apprentissage initiale.

3. Faites varier la valeur ϵ du ϵ-Greedy, et vérifiez que l'agent a le comportement adéquat.

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.1
python gridworld.py -a q -k 100 --noise 0.0 -e 0.9
```

4. Vérifiez vos résultats pour la grille `BridgeGrid`. Est-il possible de trouver une valeur de ϵ (option `-e` [...]) et du taux d'apprentissage (*learning rate*) α (option `-l` [...]) pour que l'agent trouve la politique optimale ?

```
python gridworld.py -a q -k 50 -n 0 -g BridgeGrid -e 1
```

1.7 Bras Mécanisé

Pour vérifier la généralité de votre code, vous devriez être capable d'utiliser votre agent q-learning dans un domaine différent des grilles utilisées jusqu'à présent.

1. Sans modification à votre code actuel[2], exécutez la commande suivante, pour un bras mécanisé (Fig. 8):

```
python crawler.py
```

   Si cela ne fonctionne pas, votre code est probablement trop spécifique au domaine de la grille et vous devriez le rendre plus générique.

2. Modifiez les divers paramètres d'apprentissage dans l'interface graphique apparaissant à l'exécution de `crawler.py`, et observez leurs effets sur les politiques et les actions de l'agent.

[2]Les fichiers `crawler.py` et `graphicsCrawlerDisplay.py` adaptés aux versions 3.* de Python sont disponibles sur Moodle.

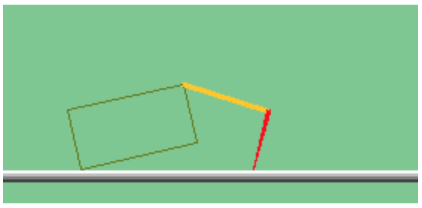


Figure 8: Bras mécanisé (`crawler`).

2 Défi RL2

2.1 PacMan et le q-learning

Pour le jeu de PacMan complet (grille de 28×31 cases), le nombre de paires (états, actions)=(s, a) possibles est si grand qu'une représentation exacte de la fonction $q(s, a)$ ne peut être parfaitement mémorisée :

- l'espace des états $\mathcal{S}$ comprend toutes les configurations possibles du jeu (position des pointes, des fantômes, de PacMan, ...),
- l'espace des actions $\mathcal{A}$ comprend les mouvements possibles de PacMan (haut, bas, gauche, droite) ainsi que la possibilité de rester immobile.

Il devient alors nécessaire de représenter la fonction de valeur $q(s, a)$, de manière approchée, à l'aide d'un nombre modéré de coefficients.

On testera cette idée sur plusieurs interfaces graphiques correspondant à des versions simplifiées du jeu de PacMan de complexité croissante : `smallGrid`, `mediumGrid`, `mediumClassic`. Ces grilles ont pour tailles respectives 5×5, 6×5, 18×9, mais certaines de ces cases ne sont pas accessibles aux agents (Pacman et fantômes).

Dans chaque cas, PacMan ne possède qu'une seule vie, et le comportement des fantômes n'est pas optimisé. L'agent PacMan effectue une action a_t à chaque instant discret t. L'état global du système (position de PacMan, des fantômes, ...) subit alors une transition résultant en une récompense définie comme suit (voir `pacman.py`): si PacMan mange une boule (ou pac-dot), il gagne 10 points; s'il gagne la partie en mangeant le dernier pac-dot du jeu, il gagne 500 points; par contre, s'il cogne un fantôme, il perd 500 points, sauf si le fantôme était mangeable, auquel cas il gagne 200 points. Un *living*

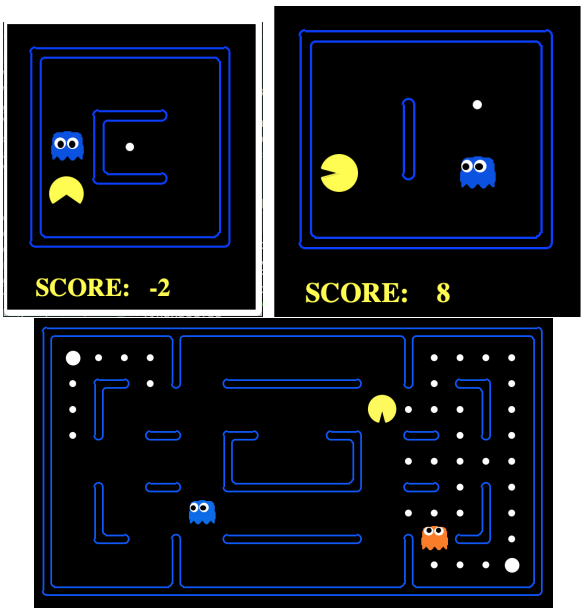


Figure 9: Small Grid, Medium Grid et Medium Classic

reward négatif (`-TIME_PENALTY`=-1) est également introduit, pour favoriser les stratégies rapides et éviter que PacMan ne tourne en rond.

PacMan jouera des parties en 2 phases:

- apprentissage, où PacMan apprendra les valeurs des cases et des actions. Cette phase se déroule sans interface graphique par défaut, pour ne pas trop ralentir le processus.
- test, où PacMan arrêtera d'apprendre (en mettant les variables `self.alpha` et `self.epsilon` à 0 pour stopper respectivement le q-learning et l'exploration) et exploitera la politique qu'il a apprise. Cette phase se déroule avec interface graphique par défaut, pour la visualisation des parties de test.

1. Utiliser votre agent q-learning développé précédemment, avec 2000 épisodes d'apprentissage (`-x 2000`) suivis de 2010 épisodes de test (`-n 2010`).

   ```
   python pacman.py -p PacmanQAgent -x 2000 -n 2010 -l smallGrid
   ```

   L'agent `PacmanQAgent` est déjà défini pour vous dans le code, et se base sur le `QLearningAgent` déjà implémenté, avec des paramètres adaptés ($\epsilon = 0.05, \alpha = 0.2, \gamma = 0.8$). Il ne faut donc pas écrire de code supplémentaire par rapport à précédemment.

   L'agent devrait s'exécuter sans erreur et à gagner au moins 80% du temps. Vous pouvez expérimenter avec différentes combinaisons des paramètres d'apprentissage avec l'option `-a`, par exemple

   ```
   [...] -a epsilon=0.1,alpha=0.3,gamma=0.7,
   ```

   ce qui permet d'accéder aux variables `self.epsilon`, `self.gamma` et `self.alpha` au sein de l'agent.

2. Visualisez 10 jeux d'entraînement (cachés par défaut) pour avoir une idée de la phase d'entraînement

   ```
   python pacman.py -p PacmanQAgent -n 10 -l smallGrid
        -a numTraining=10
   ```

3. Observez que durant l'apprentissage, des statistiques seront affichées à chaque 100 parties jouées. Comme $\epsilon > 0$ durant l'apprentissage, PacMan obtiendra de mauvais résultats même après avoir appris une bonne politique, parce qu'il exécutera occasionnellement des actions aléatoires qui pourraient l'envoyer sur un fantôme.

4. Entraînez PacMan sur un nombre croissant de parties, pour avoir une idée du nombre de parties nécessaires à l'apprentissage. Entre 1000 et 1400 parties devraient être nécessaires afin que PacMan obtienne des récompenses positives sur un segment d'une centaine d'épisodes. Vers la fin de l'apprentissage, la récompense devrait rester positive et devrait être assez élevée (entre 100 et 350).

5. Entraînez le même agent sur une grille `mediumGrid`. Dans ce cas, Pacman se débrouille moins bien, et ses récompenses restent négatives pendant l'entraînent, qui de plus est très long. Effectivement, l'espace des états est très vaste, puisque chaque configuration de jeu (position de PacMan, des fantômes, ...) est un état séparé. PacMan peut donc difficilement acquérir des connaissances sur la dangerosité d'un fantôme.

2.2 PacMan et le q-learning approximatif

Considérons un algorithme q-learning approximatif qui calcule la valeur de $q(s, a)$ selon l'équation (2).

$$q(s, a) = \sum_{i=1}^{n} w_i f_i(s, a) \tag{2}$$

1. Implémentez un agent q-learning approximatif qui apprend des poids w_i pour des caractéristiques $f_i(s, a)$ des paires (états, actions) de l'équation (2), où plusieurs états pourraient partager les mêmes caractéristiques. Écrivez votre implémentation dans la classe `ApproximateQAgent` du fichier `qlearningAgents.py`, en tenant compte des indications suivantes

   - Des fonctions $f_i(s, a)$ sont fournies dans le fichier featureExtractors.py.
   - Implémenter le vecteur de poids $\boldsymbol{w}$ comme un dictionnaire de type `util.Counter` associant des caractéristiques à des poids.
   - Mettre à jour vos vecteurs de poids de façon semblable à la mise à jour des valeurs q, en utilisant les équations (3) et (4).
   - La classe `ApproximateQAgent` hérite de la classe `QLearningAgent`. Assurez-vous que vos méthodes dans `QLearningAgent` appellent la méthode `getQValue` au lieu d'accéder directement aux valeurs q, de façon à ce que vous puissiez redéfinir cette méthode et calculer les actions en fonction de ces nouvelles valeurs q approximatives.

$$\Delta(s, a) = [R(s, a, s') + \gamma \max_{a'} q(s', a')] - q(s, a) \tag{3}$$

$$w_i \leftarrow w_i + \alpha \Delta(s, a) f_i(s, a) \tag{4}$$

2. Pour tester votre implémentation, utiliser l'agent `ApproximateQAgent` avec l'objet par défaut `IdentityExtractor`, qui associe une seule caractéristique à toutes les paires (état, action). Avec cette fonction d'extraction des caractéristiques, votre agent de q-learning approximatif devrait se comporter exactement de la même façon que l'agent `PacmanQAgent` précédent.

   ```
   python pacman.py -p ApproximateQAgent -x 2000 -n 2010
       -l smallGrid
   ```

3. Lorsque vous êtes confiants du bon fonctionnement de votre agent avec la caractéristique identité (`IdentityExtractor`), vous pouvez lancer votre agent avec l'extracteur de caractéristiques simple, afin qu'il puisse facilement apprendre à gagner:

   ```
   python pacman.py -p ApproximateQAgent
       -a extractor=SimpleExtractor -x 50 -n 60 -l mediumGrid
   ```

4. Essayez à présent avec une grille de PacMan encore plus grande

   ```
   python pacman.py -p ApproximateQAgent
        -a extractor=SimpleExtractor -x 50 -n 60 -l mediumClassic
   ```

   L'apprentissage peut prendre quelques minutes, mais devrait être réalisable.

   Si vous n'avez pas commis d'erreur dans votre implémentation, l'agent approximate q-learning devrait gagner la partie presque tout le temps, avec ces caractéristiques f_i assez simples, en seulement une cinquantaine de jeux d'entraînement.

5. Pour améliorer le comportement de votre PacMan, rajouter d'autres caractéristiques. Par exemple, tel quel PacMan évite les fantômes, même s'ils deviennent comestibles, ce qui est améliorable ...

3 Défi RL3

Le défi RL2 réalisait une progression importante en apprentissage par renforcement, en utilisant des techniques de q-learning approximatif lorsque l'espace des états est trop vaste. Il reposait sur la programmation de fonctions de valeurs f_i dans (2).

L'étape suivante est de ne plus fabriquer soi-même ces caractéristiques f_i (*feature engineering*), mais de les apprendre automatiquement.

3.1 Eléments de deep q-learning

Une solution est alors d'utiliser un réseau de neurones profond pour apprendre les fonctions de valeurs ou les q-values, selon une architecture *Deep q-Network* (DQN) illustrée à la Fig. 10.

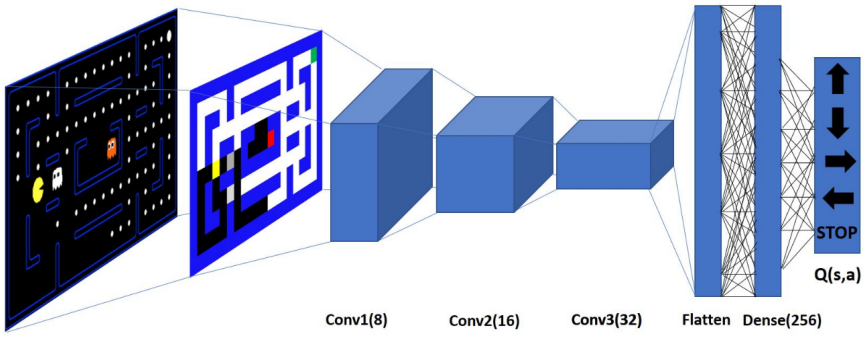


Figure 10: Diagramme d'une architecture possible d'un *Deep q-Network* (DQN), pour apprendre une fonction $q(s, a)$ à partir d'images de PacMan.

Cette architecture utilise

- comme entrée une image du jeu de PacMan; on peut également en utiliser plusieurs,
- une image équivalente, représentant de manière synthétique l'information dans l'image initiale,
- trois couches de réseaux de neurones convolutionnels (CNN) permettant d'exploiter la connectivité locale l'image,

- une couche de *flattening* et une couche pleinement connectée (*fully connected*),

- la sortie du réseau comprend toutes les actions possibles (haut, bas, gauche, droite et stop) pour l'agent.

Les couches de *Pooling* typiquement utilisées en *Deep Learning* ne sont pas forcément nécessaires, vu que la résolution des grilles de PacMan n'est pas très élevée au départ.

La politique (stratégie) π de l'agent est alors de sélectionner l'action correspondant à la valeur $q(s, a)$ la plus élevée après une *feed-forward pass* de l'état actuel s (= l'image courante du jeu) dans le réseau DQN.

Pour faire fonctionner correctement une architecture de Deep q-Learning, il faut également mettre en place, comme expliqué dans l'introduction théorique,

a) une mémoire tampon (*replay memory*) contenant un nombre N de tuples (s, a, r, s'). Un mini-batch de ces tuples sera échantillonné périodiquement (avec une période à définir),

b) un deuxième réseau de neurones, le *Target q-network*.

3.2 Implémentation de deep q-learning

- Mettre en place un Deep q-learning adapté au jeu de PacMan, en vous aidant éventuellement des indications dans le document sur Moodle (`Pacman_DQN.pdf`). Ce document correspond au code de Tycho van der Ouderaa (Univ. Amsterdam), disponible sur `GitHub` :

  `https://github.com/tychovdo/PacmanDQN`

- faites éventuellement varier le nombre de couches des réseaux, les fonctions d'activation, ...

Suggestion : utiliser `keras-rl` au lieu de `tensorflow`, ce qui facilite à la fois l'installation et la concision du code.

4 Evaluation

Pour l'évaluation des parties RL1 et RL2, déposer sur `Moodle` dans l'onglet "`Dépôt des fichiers`".

- `valueIterationAgents.py`
- `qlearningAgents.py`.

Pour la partie RL3, déposer l'entièreté des fichiers qui vous ont permis d'implémenter le deep q-learning, accompagnés d'un rapport indiquant ce que vous avez exploré parmi toutes les possibilités d'implémentation de deep q-learning.

Ce rapport peut notamment aborder les points suivants:

- Discuter du choix des "hyperparamètres" : structure des réseaux de neurones, learning rate α, discount factor γ, ϵ (si on utilise une politique ϵ-greedy).
- Quel est le gain en temps de calcul, suite à l'utilisation du Deep q-learning ?
- Etudier le comportement de l'agent PacMan avec le Deep q-learning. Il se peut qu'il ait des difficultés dans des situations qu'il a rencontrés peu souvent lors de son entraînement, par exemple des configurations de fin de partie.
  - Dans quelles situations a-t-il des difficultés ?
  - Comment améliorer son comportement dans ce cas-là ?

Si vous travaillez en groupe, indiquez clairement qui en sont les membres.

5 Pour aller plus loin . . . (optionnel)

5.1 Algorithmes d'apprentissage par renforcement

En suivant la théorie exposée au cours, implémentez votre propre agent selon les algorithmes

1. *First-visit Monte-Carlo*,
2. *SARSA*,
3. *Temporal Difference* TD(0).

5.2 Autres jeux et environnements

Il est possible d'utiliser d'autres jeux que PacMan, par exemple en utilisant l'environnement `Gymnasium` (précédemment `OpenAI gym`).

`https://gymnasium.farama.org//`

Cet environnement permet de mettre en place facilement d'autres environnements ou jeux que PacMan, en quelques lignes de code. Par exemple, pour le problème du `CartPole` (`https://gymnasium.farama.org/environments/classic_control/cart_pole//`):

```
import gym
env = gym.make("CartPole-v1")
observation = env.reset()
for _ in range(1000):
  env.render()
  action = env.action_space.sample()
  observation, reward, done, info = env.step(action)
  if done:
    observation = env.reset()
env.close()
```

Vous pouvez également tester vos algorithmes favoris sur la compétition `DeepTraffic`, pour la conduite autonome de véhicules:

- https://selfdrivingcars.mit.edu/deeptraffic/